

Experiment 1

Aim - Study of PROLOG

Introduction to Prolog

Prolog (short for "Programming in Logic") is a declarative programming language, meaning it focuses on what to solve rather than how to solve it. Developed in the early 1970s by Alain Colmerauer and Philippe Roussel, Prolog is based on formal logic, specifically predicate logic. It is widely used in artificial intelligence (AI) applications, especially in areas like natural language processing, expert systems, and knowledge representation.

Unlike languages such as C, Java, or Python, where the programmer specifies a sequence of steps to execute, Prolog allows the programmer to describe logical relationships between data. Prolog then figures out the steps needed to derive a solution, making it an excellent fit for solving problems that involve complex relationships, like reasoning and decision-making.

PROLOG stands for Programming in Logic, an idea that emerged in the early 1970s to use logic as a programming language. The early developers of this idea included Robert Kowalski at Edinburgh (on the theoretical side), Maarten van Emden at Edinburgh (experimental demonstration), and Alain Colmerauer at Marseilles (implementation). David D.H. Warren's efficient implementation at Edinburgh in the mid-1970s greatly contributed to the popularity of PROLOG.

Facts in Prolog

Facts are the simplest building blocks in Prolog. A fact is a statement about the world, which is unconditionally true. In other words, it asserts that something exists or a relationship holds without needing to be proved.

Example of Facts

```
father(john, mary).  
mother(mary, alice).
```

These are basic facts, stating:

- John is the father of Mary.
- Mary is the mother of Alice.

In Prolog, facts are written as `predicate(arguments)`., where the `predicate` represents the relation, and `arguments` are the entities involved.

- `father` and `mother` are predicates.
- `john`, `mary`, and `alice` are constants (or atomic values).

When a fact is included in the Prolog program, Prolog assumes it to be true, and it can be used in rules and queries for further inferences.

Rules in Prolog

Rules in Prolog are used to define relationships that hold true under certain conditions. A rule can be thought of as a conditional statement, similar to "if-then" statements in other languages. The body of a rule specifies the conditions that must be true for the rule to be applied.

A rule is made up of two parts:

1. **Head:** The conclusion or outcome of the rule.
2. **Body:** The conditions that must be satisfied for the head to be true.

Example of a Rule

```
grandfather(X, Y) :- father(X, Z), father(Z, Y).
```

This rule reads as "X is a grandfather of Y if X is a father of Z and Z is a father of Y."

The symbol `:-` is read as "if." So, Prolog will deduce the relationship `grandfather(X, Y)` only if the conditions `father(X, Z)` and `father(Z, Y)` are both true.

The body of the rule contains two predicates, `father(X, Z)` and `father(Z, Y)`. Prolog will attempt to satisfy both conditions in sequence when trying to determine if the rule applies.

Predicates in Prolog

A predicate is a symbol that represents a relation between terms. Predicates can be used in facts and rules. They are the essence of defining relationships in Prolog.

In the rule:

```
grandfather(X, Y) :- father(X, Z), father(Z, Y).
```

`grandfather/2` and `father/2` are predicates. The `/2` indicates that each predicate has two arguments, which is the arity of the predicate. A predicate can have any number of arguments.

In the fact:

```
father(john, mary).
```

`father/2` is the predicate, and `john` and `mary` are the arguments.

In Prolog, predicates are used to express relationships. They can represent complex relations like "is a parent of," "is a sibling of," "is a friend of," and so on. These relationships form the core of Prolog's logic system.

Clauses in Prolog

A clause in Prolog is either a fact or a rule. It is a fundamental part of any Prolog program and provides a way to express logical relationships.

A Prolog program consists entirely of clauses, which can be:

1. **Facts:** Unconditional truths.
2. **Rules:** Conditional truths (used for inference).

Example of a Clause

```
father(john, mary). % Fact  
grandfather(X, Y) :- father(X, Z), father(Z, Y). % Rule
```

A fact simply declares something true, while a rule expresses a relationship between facts that Prolog can use to deduce new facts. A Prolog program consists of a collection of such clauses. When querying Prolog, it will attempt to match the query with facts and use rules to infer additional facts.

Features of Prolog

Prolog has several distinct features that make it particularly suited for specific kinds of tasks, especially in AI and logic-based applications:

1. Declarative Nature

In Prolog, you describe what the problem is and let the system figure out how to solve it. This contrasts with procedural languages, where you explicitly define the sequence of operations. This makes Prolog particularly suited for knowledge representation, reasoning, and problem-solving tasks.

2. Automatic Backtracking

Prolog automatically searches for solutions by trying different paths and backtracking when it reaches a dead end. This allows Prolog to explore multiple possible solutions without the programmer needing to manually implement backtracking logic.

3. Pattern Matching

Prolog uses pattern matching to check if a goal (query) matches with facts or rules in the knowledge base. It tries to unify the terms in the query with terms in the facts or rules.

4. Recursion

Prolog's approach is heavily recursive, which means that problems are often solved by breaking them down into smaller, self-similar subproblems. Prolog's rules and queries often define a solution in terms of simpler instances of the same problem.

5. Unification

Unification is a central concept in Prolog. It refers to the process of making two terms identical by finding an appropriate substitution for their variables. This allows Prolog to match queries with facts and rules dynamically.

6. Logical Variables

Prolog uses variables that can be instantiated during the execution of the program. When Prolog tries to satisfy a query, it will substitute values for variables to make the query true.

7. Non-determinism

Can return multiple solutions to a problem by exploring different possible answers.

8. Built-in Predicates

Provides pre-defined predicates for tasks like arithmetic, list manipulation, and more.

9. Dynamic Database

Allows the addition and removal of facts and rules during program execution (assert/retract).

10. Constraint Logic Programming (CLP)

Extends Prolog for solving problems with constraints, such as optimization or scheduling.

11. Natural Language Processing (NLP)

Well-suited for parsing and interpreting languages, often used in AI and linguistics.

12. Built-in Lists and Data Structures

Native support for lists and other common data structures for handling collections of terms.

13. No Side Effects

Purely declarative, with no hidden state or side effects during computation.

14. Interactivity

Supports interactive development where users define and query facts, rules, and conditions in real-time.

Symbolic Language in Prolog

Prolog is often called a symbolic language because it deals with symbolic terms rather than raw data. In contrast to numeric computation or simple data manipulation, Prolog focuses on relationships between symbolic terms.

- **Atoms:** These are basic symbols in Prolog. An atom can be a simple constant like `john` or a more complex structure like `'hello world'`.
- **Variables:** Represent unknown values that Prolog will attempt to assign values to during execution. Variables are usually written in uppercase, like `X`, `Y`, or `Person`.
- **Predicates:** Symbolic names for relations between terms, such as `father/2`, `likes/2`, etc.
- **Terms:** A term in Prolog can be an atom, a variable, or a compound term (a structure). Compound terms consist of a functor and a list of arguments, e.g., `father(john, mary)` is a compound term.

By using these symbols, Prolog allows for logical reasoning based on abstract relationships and structures, making it suitable for tasks that require symbol manipulation, such as language processing or expert systems.

PROLOG is a programming language for symbolic, non-numeric computation. It is especially well suited for solving problems that involve objects and relations between objects. Features like this make PROLOG a powerful language for Artificial Intelligence (AI) and non-numerical programming.

Symbolic Nature

1. It uses symbols to represent facts, rules, and queries in its knowledge base.
2. Symbols in Prolog include constants, variables, predicates, and operators.
3. Logical relationships are expressed through symbolic rules, using operators like `:-` (if) and `,` (and).
4. Prolog's symbolic nature makes it ideal for applications like artificial intelligence and knowledge representation.
5. The combination of symbols and logical inference allows Prolog to solve complex problems through reasoning.

Symbols Used in Prolog

Symbol	Name	Usage
<code>.</code>	Full stop	Used to terminate clauses or queries.
<code>:-</code>	Rule operator	Denotes "if" in rules, separating head from body.
<code>,</code>	Comma	Represents logical AND in queries or rule bodies.
<code>;</code>	Semicolon	Represents logical OR in queries or rules.
<code>=</code>	Equality operator	Used to test if two terms are the same.
<code>\=</code>	Inequality operator	Tests if two terms are not the same.

<code>==</code>	Strict equality	Tests if two terms are identical without bindings.
<code>\==</code>	Strict inequality	Tests if two terms are not identical.
<code>is</code>	Arithmetic evaluation	Evaluates arithmetic expression.
<code>=<</code>	Less than or equal	Used for arithmetic comparisons.
<code><</code>	Less than	Used for arithmetic comparisons.
<code>>=</code>	Greater than or equal	Used for arithmetic comparisons.
<code>></code>	Greater than	Used for arithmetic comparisons.

Meta Programming in Prolog

Meta-programming in Prolog is the practice of writing programs that treat other programs as data. This allows Prolog to generate or modify rules and facts at runtime, providing powerful capabilities for dynamic reasoning and reflection.

Key Concepts

- **Program as Data:** Prolog treats code as data, allowing the program to manipulate or inspect other parts of itself during execution.
- **Dynamic Predicates:** Prolog allows predicates to be asserted (added) or retracted (removed) at runtime, enabling dynamic modifications of the knowledge base.
- **Term Construction and Unification:** Prolog allows the dynamic construction of terms and the unification of terms, enabling flexible manipulation of predicates and goals.
- **The `=..` Operator (Univ):** This operator allows you to convert a term into a list of its components or vice versa, providing a way to dynamically generate or inspect predicates.
- **Creating and Modifying Rules Dynamically:** Prolog allows the creation or modification of rules at runtime, enabling the program to adapt and change based on new information.
- **Reflection:** Prolog can introspect its own code to retrieve or manipulate predicates, facts, and clauses, enabling self-examination and modification of the program.
- **Goal Construction:** Prolog supports dynamic creation of goals at runtime, enabling flexible query generation and execution.
- **Higher-Order Predicates:** Prolog allows predicates to accept other predicates as arguments, supporting higher-order logic and enabling more abstract programming.
- **Meta-Interpretation:** Prolog programs can interpret or execute other Prolog programs, allowing for custom reasoning or translation systems.

Example of Meta-Programming

```
assert(father(john, mary)).
```

This adds a fact `father(john, mary)` to the knowledge base during execution. Similarly, Prolog has predicates like `retract/1`, `assertz/1`, and `asserta/1` for manipulating the knowledge base.

Meta-programming is used in applications like:

- Dynamic knowledge base updates: Adding or removing facts during program execution.
 - Creating interpreters: Building systems that interpret or execute other Prolog programs.
-

Expert Systems in Prolog

An expert system is a software application that emulates the decision-making ability of a human expert. Prolog is particularly well-suited for developing expert systems because it can easily represent complex relationships and make logical inferences based on a set of rules.

Components of Expert Systems

1. Knowledge Base

The knowledge base consists of facts and rules that represent the expert knowledge in a specific domain. This knowledge is encoded in Prolog as facts (assertions about the world) and rules (logical relationships that define how knowledge is derived).

Example: Facts could describe relationships, like `parent(john, mary)` (John is Mary's parent), while rules could describe derived knowledge, like `grandparent(X, Y) :- parent(X, Z), parent(Z, Y)` (X is a grandparent of Y if X is a parent of Z and Z is a parent of Y).

2. Inference Engine

The inference engine is the component that applies logical reasoning to the knowledge base to draw conclusions. In Prolog, this is done using backtracking and unification. It attempts to match queries with the knowledge base and deduces new information by applying rules. It processes queries by finding relevant facts and applying rules to infer answers.

3. User Interface

A user interface allows the system to interact with the user, typically asking questions and interpreting their answers to form new facts. This could involve interactive prompts or structured inputs. Prolog's interactive nature, where you can query the system and receive answers, fits well as an interface for expert systems.

4. Explanation Facility

An explanation facility helps the system explain its reasoning process to the user. In Prolog, this could involve explaining how a conclusion was reached by showing the chain of facts and rules that led to the result. This can be implemented by tracing the rules and facts used to derive a particular conclusion.

5. Working Memory

The working memory stores facts that are known to be true at a given moment in time. This memory is updated as new facts are discovered during the reasoning process.

Example Expert System

For instance, an expert system for diagnosing medical conditions might include rules like:

fever(yes) :- temperature(>100).

cough(yes) :- throat_infection(yes).

By querying the expert system, the system can infer the diagnosis by combining rules and facts, similar to how a human expert would deduce the best course of action.

Installation Steps for SWI-Prolog

Step 1: Visit Website

Visit swi-prolog.org website using any web browser.



Step 2: Navigate to Download

Click on **Download** which is adjacent to Home. A dropdown list will appear, then click on **SWI-Prolog**.



Step 3: Select Stable Release

A new webpage will open. Click on **Stable release**.



Step 4: Choose Platform

After clicking on stable release, a new webpage will open which will contain stable versions of Prolog for different platforms. Under binaries, there are two stable releases for Windows:

- SWI-Prolog 8.4.0-1 for Microsoft Windows (64 bit)
- SWI-Prolog 8.4.0-1 for Microsoft Windows (32 bit)

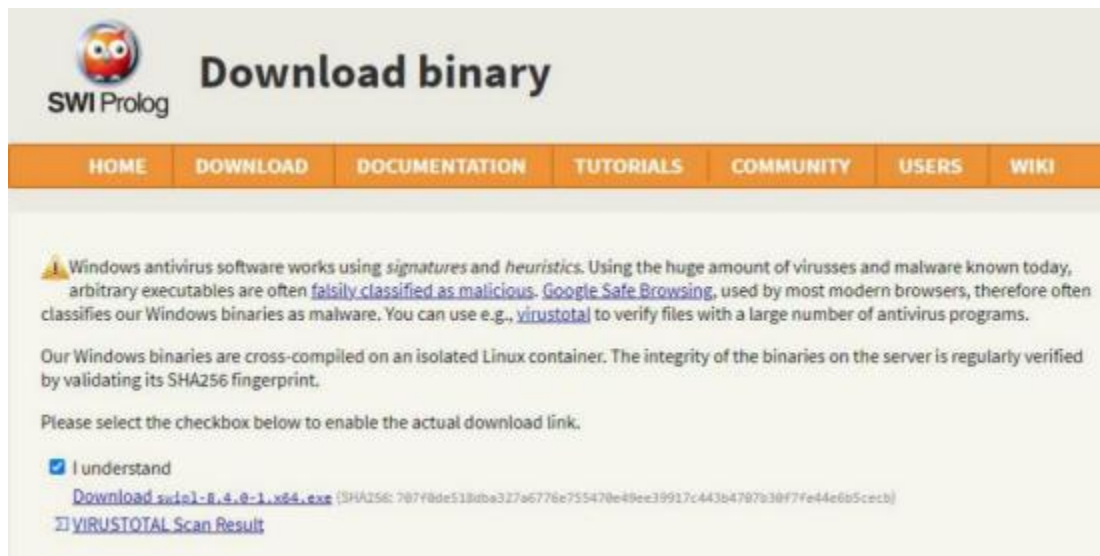
Click on the one as per your system configuration. Let's take the one for 64-bit operating system.



Step 5: Download Executable

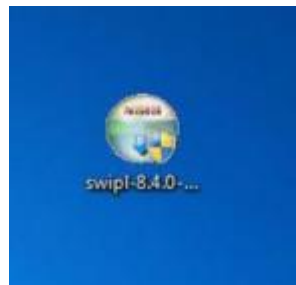
After clicking on SWI-Prolog 8.4.0-1 for Microsoft Windows (64 bit), a new webpage will open. Check the "I understand" checkbox to make the download link active. Then click on the download link.

Downloading of the executable file will start shortly. It is a small 11.9 MB file that will hardly take a minute.



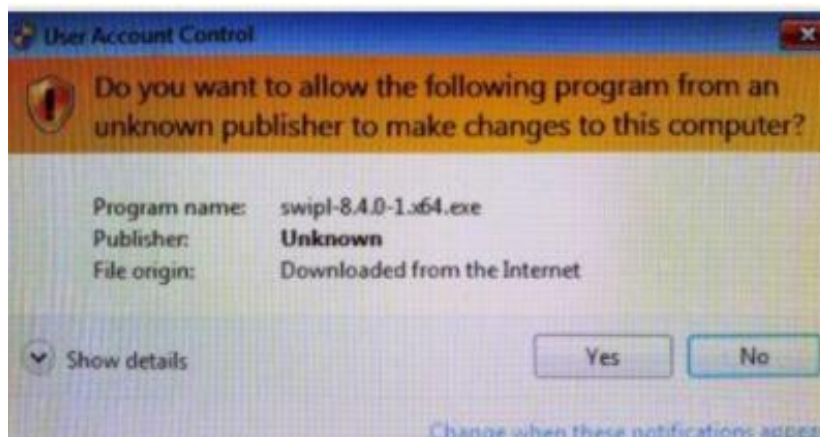
Step 6: Run Installation File

Check for the executable file in downloads in your system and run it.



Step 7: Allow Changes

It will prompt confirmation to make changes to your system. Click on **Yes**.



Step 8: Begin Setup

Setup screen will appear. Click on **Next**.



Step 9: Accept License Agreement

The next screen will be of License Agreement. Click on **I Agree**.



Step 10: Installation Options

After that, there will be a screen of installing options:

- Check the box for **Add swipl to the system path for all users**
- Check the box for **Create a desktop icon**
- Click on the **Next** button



Step 11: Choose Installation Location

The next screen will be of installing location. Choose the drive which will have sufficient memory space for installation. It needs only a memory space of 50 MB.



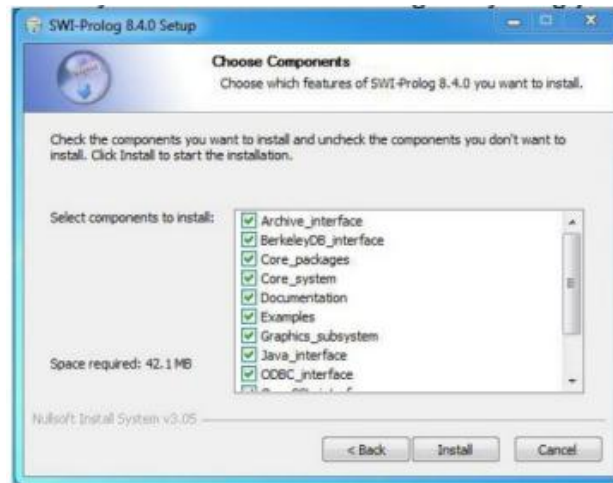
Step 12: Start Menu Folder

Next screen will be of choosing Start menu folder. Don't change anything, just click on **Next** button.



Step 13: Choose Components

This last screen is of choosing components. All components are already marked, so don't change anything. Just click on **Install** button.



Step 14: Installation Process

After this, the installation process will start and will hardly take a minute to complete the installation.



Step 15: Complete Installation

Click on **Finish** after the installation process is complete.



Step 16: Verify Installation

SWI Prolog is successfully installed on the system and an icon is created on the desktop.



Step 17: Launch Software

Run the software and see the interface.



Congratulations! At this point, you have successfully installed SWI-Prolog on your Windows system.

Hello World Program in Prolog

Code

Save the following Prolog code in a file named `hello.pl`:

```
hello :-  
    write('Hello World!'), nl.
```

Running the Program

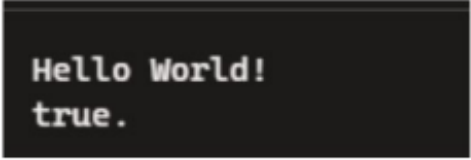
1. Load the program in your Prolog interpreter with:

```
?- consult('hello.pl').
```

2. Run the predicate `hello`:

```
?- hello.
```

Output



```
Hello World!  
true.
```

The `nl` predicate is used to move to a new line after printing. This demonstrates Prolog's ability to interact with the outside world by printing to the console.

Direct Execution

Alternatively, you can execute directly without saving to a file:

```
?- write('Hello World!'), nl.  
Hello World!  
true.
```


Viva Voce - 1

1. What are facts, rules, and queries in PROLOG?

- Facts represent known information. Example: `parent(john, mary)`.
- Rules define relationships between facts. Example: `grandparent(X, Y) :- parent(X, Z), parent(Z, Y)`.
- Queries are used to check information. Example: `?- grandparent(john, mary)`.

2. What are predicates and clauses in PROLOG?

- A predicate defines a relation between objects. Example: `likes(john, pizza)`.
- A clause is a single unit in PROLOG, either a fact, a rule, or a query.

3. What is an expert system in PROLOG?

An expert system is a program that mimics human decision-making using a knowledge base and inference rules. PROLOG is widely used for expert systems due to its logical programming nature.

4. What are some key features of PROLOG?

- Declarative language (focuses on what to solve, not how)
- Symbolic and Logical Processing for AI applications
- Backtracking to find multiple solutions
- Built-in pattern matching and recursion

5. How do you install and run a PROLOG program?

- Install SWI-PROLOG from www.swi-prolog.org
- Write a PROLOG program (`hello.pl`) with `write('Hello, World!')`.
- Run using `swipl -f hello.pl` or load with `consult('hello.pl')`. and execute

6. What is Meta-Programming in PROLOG?

Meta-programming in PROLOG refers to writing programs that manipulate or reason about other programs. It allows the use of code as data, enabling advanced AI applications like self-modifying code.

7. What are the main symbols used in PROLOG, and what do they represent?

- `.` (Period): Ends a statement
- `:-` (Colon-Dash): Defines a rule (if-then relationship)
- `,` (Comma): Represents logical AND
- `;` (Semicolon): Represents logical OR

Experiment 2

Aim:

Write simple facts for the statements using PROLOG

Problem Statements:

Write Prolog facts for the following statements:

1. Ram likes mango.
 2. Seema is a girl.
 3. Bill likes Cindy.
 4. Rose is red.
 5. John owns gold.
-

Theory:

Facts in Prolog

Facts are the fundamental building blocks of a Prolog program. They represent basic assertions about the world that are unconditionally true. A fact consists of a predicate followed by one or more arguments enclosed in parentheses, and it always ends with a period (.).

Syntax of Facts

```
predicate(argument1, argument2, ...).
```

Components

- **Predicate:** The relation or property being defined (e.g., likes, owns, red)
- **Arguments:** The objects or entities involved in the relation (e.g., ram, mango, john)
- **Period:** Every fact must end with a period (.)

Naming Conventions

- **Atoms (constants):** Start with lowercase letters (e.g., ram, mango, bill)
- **Variables:** Start with uppercase letters (e.g., X, Person, Item)
- **Predicates:** Start with lowercase letters (e.g., likes, owns, girl)

Types of Facts

1. Unary Facts (Single Argument)

Represent properties of a single entity:

```
girl(seema).  
red(rose).
```

2. Binary Facts (Two Arguments)

Represent relationships between two entities:

```
likes(ram, mango).  
owns(john, gold).
```

3. N-ary Facts (Multiple Arguments)

Can represent complex relationships with multiple entities:

```
parent(john, mary, 1990).
```

Procedure

Step 1: Create a New File

Open a text editor and create a new file named exp2.pl

Step 2: Define the Facts

Write the Prolog facts in the file according to the problem statements. Each fact must:

- Start with a lowercase predicate name
- Have arguments in parentheses
- End with a period (.)

Step 3: Save the File

Save the file in your working directory with the .pl extension.

Step 4: Open SWI-Prolog

Launch the SWI-Prolog interpreter on your system.

Step 5: Load the Program

In the Prolog interpreter, load your file using the consult command:

```
?- consult('exp2.pl').
```

Or use the shorthand notation:

```
?- ['exp2.pl'].
```

Step 6: Verify Loading

Prolog should respond with `true.` if the file loaded successfully. If there are syntax errors, Prolog will display error messages.

Step 7: Test the Facts

Query the facts to verify they are correctly stored in the knowledge base. Use various query patterns to test different aspects.

Code

```
% Facts representing the
given statements
likes(ram, mango).
girl(seema).
likes(bill, cindy).
red(rose).
owns(john, gold).
```

Explanation of Code

- `likes(ram, mango).` - Defines that Ram likes mango
- `girl(seema).` - Defines that Seema is a girl
- `likes(bill, cindy).` - Defines that Bill likes Cindy
- `red(rose).` - Defines that Rose is red
- `owns(john, gold).` - Defines that John owns gold

Running the Program

Step 1: Save the Code

Save the above facts in a file named `exp2.pl`

Step 2: Load the Program

Open SWI-Prolog and load the file:

```
?- consult('exp2.pl').
true.
```

Step 3: Query the Facts

You can now query these facts to verify them.

Output and Queries

Query 1: Checking likes(ram, mango)

```
?- likes(ram, What).  
What = mango.  
  
?- likes(Who, mango).  
Who = ram.  
  
?- likes(ram, mango).  
true.  
  
?- likes(seema, mango).  
false.
```

Explanation:

- likes(ram, What). - Asks "What does Ram like?" Returns mango
 - likes(Who, mango). - Asks "Who likes mango?" Returns ram
 - likes(ram, mango). - Checks if "Ram likes mango" Returns true
 - likes(seema, mango). - Checks if "Seema likes mango" Returns false (not in knowledge base)
-

Query 2: Checking girl(seema)

```
?- girl(Who).  
Who = seema.  
  
?- girl(seema).  
true.  
  
?- girl(ram).  
false.
```

Explanation:

- girl(Who). - Asks "Who is a girl?" Returns seema
- girl(seema). - Checks if "Seema is a girl" Returns true

- `girl(ram).` - Checks if "Ram is a girl" Returns `false`
-

Query 3: Checking `likes(bill, cindy)`

```
?- likes(bill, What).  
What = cindy.  
  
?- likes(Who, cindy).  
Who = bill.  
  
?- likes(bill, cindy).  
true.  
  
?- likes(ram, cindy).  
false.
```

Explanation:

- `likes(bill, What).` - Asks "What does Bill like?" Returns `cindy`
 - `likes(Who, cindy).` - Asks "Who likes Cindy?" Returns `bill`
 - `likes(bill, cindy).` - Checks if "Bill likes Cindy" Returns `true`
 - `likes(ram, cindy).` - Checks if "Ram likes Cindy" Returns `false`
-

Query 4: Checking `red(rose)`

```
?- red(What).  
What = rose.  
  
?- red(rose).  
true.  
  
?- red(mango).  
false.
```

Explanation:

- `red(What).` - Asks "What is red?" Returns `rose`
- `red(rose).` - Checks if "Rose is red" Returns `true`
- `red(mango).` - Checks if "Mango is red" Returns `false`

Query 5: Checking owns(john, gold)

```
?- owns(john, What).  
What = gold.
```

```
?- owns(Who, gold).  
Who = john.
```

```
?- owns(john, gold).  
true.
```

```
?- owns(bill, gold).  
false.
```

Explanation:

- owns(john, What). - Asks "What does John own?" Returns gold
- owns(Who, gold). - Asks "Who owns gold?" Returns john
- owns(john, gold). - Checks if "John owns gold" Returns true
- owns(bill, gold). - Checks if "Bill owns gold?" Returns false

Viva Voce - 2

1. How do you define a simple fact in PROLOG?

A fact in PROLOG represents a basic assertion about some entity or relationship. It follows the syntax `fact_name(argument1, argument2, ...)`. The fact must start with a lowercase letter and end with a period. Example: `likes(ram, mango).`

2. How would you write the given statements as facts in PROLOG?

The given statements can be written as:

- `likes(ram, mango).`
- `girl(seema).`
- `likes(bill, cindy).`
- `red(rose).`
- `owns(john, gold).`

3. What happens if we enter a fact in PROLOG without a period (.) at the end?

PROLOG will not recognize the input as a complete statement and will wait for further input, showing an "unexpected end-of-file" or "incomplete clause" error. The period is essential to terminate the fact.

4. How can we query PROLOG to check if a fact is true?

We can use a query with a question mark (?-), like `?- likes(ram, mango).` If the fact exists in the knowledge base, PROLOG will return true. Otherwise, it will return false.

5. Can PROLOG infer new facts from existing ones?

No, PROLOG does not infer new facts automatically unless explicitly defined using rules. Facts only store direct knowledge. Rules are needed to derive new knowledge from existing facts.

6. What happens if we query PROLOG for a fact that is not present in the knowledge base?

PROLOG will return false, indicating that the fact is not known. Example: `?- likes(john, mango).` would return false if this fact was not defined.

Experiment 3

Aim:

Write predicates, one converts centigrade temperatures to Fahrenheit, the other checks if a temperature is below freezing using PROLOG

Theory:

- **Predicates in Prolog** - A predicate in Prolog defines a relationship or property between objects. Predicates can represent both facts (unconditionally true statements) and rules (conditional statements with logical relationships).

- **Arithmetic in Prolog** - Prolog provides the `is` operator for arithmetic evaluation:

$$F \text{ is } C * 9/5 + 32.$$

- **Temperature Conversion Formula -**

Celsius to Fahrenheit:

$$F = C \times (9/5) + 32$$

Examples:

- $0^{\circ}\text{C} = 32^{\circ}\text{F}$ (freezing point of water)
- $100^{\circ}\text{C} = 212^{\circ}\text{F}$ (boiling point of water)
- $20^{\circ}\text{C} = 68^{\circ}\text{F}$ (room temperature)
- $-40^{\circ}\text{C} = -40^{\circ}\text{F}$ (same in both scales)

- **Freezing Point -**

The freezing point of water is:

- 0°C in Celsius
- 32°F in Fahrenheit

Procedure

Step 1: Create a New File

Open a text editor and create a new file named `exp3.pl`

Step 2: Define the Conversion Rule

Write the `c_to_f/2` predicate that takes Celsius as input and calculates Fahrenheit:

$$\text{c_to_f}(C, F) \text{ :- } F \text{ is } C * 9/5 + 32.$$

Step 3: Define the Freezing Check Rule

Write the `freezing/1` predicate that checks if a Fahrenheit temperature is below 32°F:

```
freezing(F) :- F < 32.
```

Step 4: Save the File

Save the file in your working directory with the `.pl` extension.

Step 5: Open SWI-Prolog

Launch the SWI-Prolog interpreter on your system.

Step 6: Load the Program

In the Prolog interpreter, load your file using the `consult` command:

```
?- consult('exp3.pl').
```

Step 7: Test the Predicates

Query the predicates to verify they work correctly. Test with various temperature values.

Code

```
% Temperature conversion predicate: Celsius to Fahrenheit
c_to_f(C, F) :- F is C * 9/5 + 32.

% Freezing check predicate: checks if temperature is below 32°F
freezing(F) :- F < 32.
```

Output -

```
[mait@fedora MAIT]$ swipl Exp3.pl

Welcome to SWI-Prolog (threaded, 64 bits, version 9.0.4)

SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- freezing(30).
true.

?- freezing(36).
false.

?- freezing(20).
true.

?- c_to_f(20,F).
F = 68.

?- c_to_f(-25,F).
F = -13.

[mait@fedora MAIT]$ |
```

Viva Voce - 3

1. How do you define a predicate in PROLOG?

A predicate in PROLOG represents a relation between objects and is used to define facts or rules. It follows the syntax `predicate_name(arguments)`. For rules, we use `head :- body`, where the head is true if the body conditions are satisfied.

2. How do you convert a temperature from Centigrade to Fahrenheit in PROLOG?

We define a rule using the formula `c_to_f(C, F) :- F is C * 9/5 + 32`. This takes a Celsius value `C` and calculates the Fahrenheit value `F` using the `is` operator for arithmetic evaluation.

3. How do you check if a temperature is below freezing (0°C or 32°F) in PROLOG?

We can define a rule like `freezing(F) :- F < 32`. This checks if the Fahrenheit temperature `F` is below 32°F and returns true if it is, false otherwise.

4. How do you query PROLOG to convert 25°C to Fahrenheit?

We use the query `?- c_to_f(25, F)`. PROLOG will evaluate the rule and return `F = 77`.

5. What will be the output of the query `?- freezing(-5)` in PROLOG?

Since -5°F is below 32°F (freezing point), PROLOG will return true.

6. What does the `is` keyword do in PROLOG calculations?

The `is` keyword is used for arithmetic operations in PROLOG. It evaluates the right-hand expression and unifies the result with the left-hand variable. Example: `F is C * 9/5 + 32`.

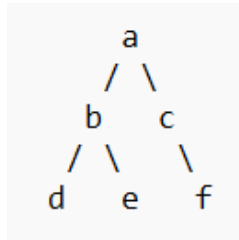
7. Can PROLOG automatically convert data types like strings and numbers in arithmetic operations?

No, PROLOG does not perform implicit type conversions. If a value is not a number, arithmetic operations will result in an error. All arithmetic operands must be numeric.

Experiment 4

Aim - Write a program to implement Breadth First Search Traversal

Tree -



Theory –

Breadth First Search is a graph traversal algorithm that explores all the vertices of a graph level by level. It visits all the neighbors of a node before moving to the next level neighbors.

1. **Level-wise Exploration:** BFS explores nodes in layers - first the starting node, then all nodes at distance 1, then distance 2, and so on.
2. **Queue-based Approach:** BFS uses a queue data structure (First In First Out) to keep track of nodes to be explored. In Prolog, we simulate this using lists.
3. **Shortest Path:** In an unweighted graph, BFS guarantees finding the shortest path between the start and goal nodes in terms of the number of edges.
4. **Visited List:** To avoid cycles and infinite loops, BFS maintains a list of visited nodes.

Procedure –

Step 1: Create a New File

Open a text editor and create a new file named bfs.pl

Step 2: Define the Tree Structure

Write facts to represent nodes and their connections

Step 3: Implement BFS Logic

Write the BFS algorithm using recursion and list processing

Step 4: Add User Interface

Create predicates to take start and goal nodes as input

Step 5: Save the File

Save the file in your working directory

Step 6: Load and Run

Open SWI-Prolog and consult the file:

```
?- consult('bfs.pl').
```

Step 7: Enter Input

Enter start node and goal node when prompted

Step 8: View Output

Observe the BFS path or "No path found" message

Code –

```
% Define the tree structure - MODIFY HERE
tree(a, [b, c]).
tree(b, [d, e]).
tree(c, [f]).
tree(d, []).
tree(e, []).
tree(f, []).

% BFS traversal to find the path between two nodes
bfs(Start, Goal, Path) :-
    bfs_helper([[Start]], Goal, Path).

% Base case: If the goal is found in the first path, return it
bfs_helper([[Goal|Visited]|_], Goal, [Goal|Visited]).

% Recursive case: Explore the next level (children of the current node)
bfs_helper([Path|Paths], Goal, FinalPath) :-
    extend(Path, NewPaths),
    append(Paths, NewPaths, Paths1),
    bfs_helper(Paths1, Goal, FinalPath).

% Extend the current path by adding the neighbors of the current node
(levelwise)
extend([Node|Path], NewPaths) :-
    findall([NewNode, Node|Path],
        (tree(Node, Neighbors),
         member(NewNode, Neighbors),
         \+ member(NewNode, Path)),
        NewPaths).
```

```
NewPaths).
```

```
% Predicate to read input from the user and run BFS level-wise traversal
```

```
run_bfs :-
```

```
    write('Enter start node: '), read(Start),
```

```
    write('Enter goal node: '), read(Goal),
```

```
    ( bfs(Start, Goal, Path) ->
```

```
        reverse(Path, ReversedPath), % Reverse to get the correct path order
```

```
        format('BFS Path from ~w to ~w: ~w~n', [Start, Goal, ReversedPath])
```

```
    ; format('No path found between ~w and ~w.~n', [Start, Goal])
```

```
    ).
```

```
% Example query to run the program
```

```
:- initialization(run_bfs).
```

Output –

```
?- consult('bfs.pl').  
Enter start node: a.  
Enter goal node: | : f.  
BFS Path from a to f: [a,c,f]  
true.
```

```
?- consult('bfs.pl').  
Enter start node: a.  
Enter goal node: | : f.  
BFS Path from a to f: [a,c,f]  
true.
```

```
?- consult('bfs.pl').  
Enter start node: b.  
Enter goal node: f.  
No path found between b and f.
```

Experiment 5

Aim - Write a program to remove punctuations from the given string

Theory –

Natural language text often contains punctuation symbols such as commas, periods, question marks, exclamation marks, quotes, and other non-alphanumeric characters. These characters are useful for human readability but can create noise for many NLP and text-processing tasks where only words and numbers are required. Strings can be handled as lists of character codes, which makes it convenient to traverse each character and filter unwanted ones using recursive predicates. By checking the character type (alphanumeric, space, punctuation, etc.), we can construct a new list that keeps only allowed characters and discards the rest.

- Represent the input string as a list of character codes.
- Use recursion to process one character at a time.
- Use built-in predicates like `char_type/2` to test whether a character is alphanumeric or whitespace.
- Convert the filtered character codes back to a Prolog string for display.

Procedure –

Step 1: Create a New File

Open a text and create a new file named `exp5.pl`.

Step 2: Include Required Library

At the top of the file, include the `readutil` library because it provides the `read_line_to_codes/2` predicate used to read an entire line of input from the user.

Step 3: Define the Entry Point

Use the:- `initialization(main).` directive so that the `main/0` predicate runs automatically when the program is executed from the command line.

Step 4: Implement the `remove_punct` Predicate

Define a recursive predicate `remove_punct/2` that:

- Takes a list of character codes as input.
- Keeps only alphanumeric and space characters.

Step 5: Implement the main Predicate

In `main/0`:

- Prompt the user to enter a string.
- Read the line using `read_line_to_codes/2`.

Step 6: Save the File

Save the file as exp5.pl in your working directory.

Step 7: Run the Program

Open a terminal, navigate to the directory containing exp5.pl, and run:

- `swipl exp5.pl`

Code –

```
:- use_module(library(readutil)). % Needed for
read_line_to_codes/2

:- initialization(main).           % Runs main automatically

% Remove punctuation from a list of character codes
remove_punct([], []).

remove_punct([Char|Rest], Filtered) :-
    (   (char_type(Char, alnum) ; char_type(Char, space))
    ->   Filtered = [Char|FilteredRest]
    ;   Filtered = FilteredRest
    ),
    remove_punct(Rest, FilteredRest).

% Main predicate to run the program
main :-
    write('Enter a string: '),
    read_line_to_codes(user_input, InputCodes),
    remove_punct(InputCodes, FilteredCodes),
    string_codes(FilteredString, FilteredCodes),
    write('Filtered string: '), write(FilteredString), nl.
```

Output –

```
?- swipl exp5.pl
Enter a string: !!!!!??!
Filtered string:

Enter a string: Kunal! 9!?7 AI! ML
Filtered string: Kunal 9 7 AI ML

Enter a string: Hello, World! This is Prolog-123.
Filtered string: Hello World This is Prolog123
```

Viva Voce - 5

1. **What is the objective of this experiment?**

To develop a Prolog program that removes punctuation marks from a user-entered string, keeping only alphanumeric characters and spaces.

2. **What is the objective of this experiment?**

Develop Prolog program to remove punctuation from input strings, keeping only alphanumeric characters and spaces for text preprocessing in NLP applications. (18 words)

3. **Why use read_line_to_codes/2 instead of read/1?**

Reads entire line including spaces as character codes, perfect for string processing. read/1 parses Prolog terms, not raw text input. (19 words)

4. **Role of char_type/2 predicate?**

Classifies character codes as alphanumeric (alnum) or space (space). Filters out punctuation by keeping only allowed character types. (17 words)

5. **How does program decide which characters to remove?**

Keeps characters if alnum OR space, discards all others (punctuation, symbols). Uses conditional logic in recursive filtering. (17 words)

6. **Data structure for string representation?**

List of character codes (integers). Prolog strings internally represented as code lists for efficient recursive processing. (15 words)

7. **Explain base case of remove_punct/2?**

remove_punct([], []). Empty input list produces empty output list, terminating recursion naturally when all characters processed. (14 words)

Experiment 6

Aim - Write a program to sort the sentence in alphabetical order

Theory –

Sorting sentences alphabetically involves splitting text into individual words, sorting those words lexicographically, and recombining them into a sentence. Prolog excels at list manipulation and provides built-in predicates for string splitting, sorting, and concatenation.

- `split_string/4`: Splits sentence on whitespace into list of words.
- `sort/2`: Sorts list in ascending alphabetical order (case-sensitive).
- `atomic_list_concat/3`: Joins sorted words with spaces into final sentence.
- `read_line_to_string/2`: Reads complete line input as string.

Procedure –

Step 1: Create New File

Open text editor, create `exp6.pl`.

Step 2: Define Auto-Run

Add `:- initialization(main).` for automatic execution.

Step 3: Implement Word Splitting

Define `split_sentence/2` using `split_string(Sentence, " ", "", WordList)` to split on spaces.

Step 4: Implement Sorting Logic

Create `sort_sentence/2` that splits, sorts words with `sort/2`, concatenates with `atomic_list_concat/3`.

Step 5: Create Main Predicate

In `main/0`: prompt input, read line, sort, print sorted sentence.

Step 6: Save File

Save as `exp6.pl` in working directory.

Step 7: Execute Program

Run `swipl exp6.pl` from terminal.

Code –

```
:- initialization(main).  
  
% Convert a sentence into a list of words  
split_sentence(Sentence, WordList) :-  
    split_string(Sentence, " ", "", WordList).  
  
% Sort the list of words  
sort_sentence(Sentence, SortedSentence) :-  
    split_sentence(Sentence, WordList),  
    sort(WordList, SortedWords),  
    atomic_list_concat(SortedWords, ' ', SortedSentence).  
  
% Main function to take input and display output  
main :-  
    write('Enter a sentence: '),  
    read_line_to_string(user_input, Sentence),  
    sort_sentence(Sentence, SortedSentence),  
    write('Sorted sentence: '), write(SortedSentence), nl.
```

Output –

```
?- sort_sentence("Kunal AI Prolog Lab", Sorted).  
Sorted = "AI Kunal Lab Prolog" .
```

```
?- split_sentence("Kunal loves Prolog", Words).  
Words = ["Kunal","loves","Prolog"] .
```

```
?- sort(["Zunal","Kunal","Aunal"], Sorted).  
Sorted = ["Aunal","Kunal","Zunal"] .
```

```
?- main.  
Enter a sentence: Kunal is learning Prolog!  
Sorted sentence: Kunal Prolog is learning!  
true.
```

Viva Voce - 6

1. What is the purpose of a Prolog program to sort a sentence?

The purpose of this program is to take a sentence as input, break it into words, and arrange them in sorted order. Prolog uses built-in predicates like `split_string/4` and `msort/2` for this task. The sorted sentence is returned as output while maintaining punctuation if needed. This helps in understanding Prolog's list processing capabilities.

2. How does Prolog handle string manipulation in the sorting process?

`split_string/4` predicate is used to break a sentence into words. The words are stored as a list, which is then sorted using `msort/2`. Finally, the sorted words are concatenated back into a string using `atomic_list_concat/3`.

3. What is the role of the `msort/2` predicate in sorting a sentence?

The `msort/2` predicate in Prolog is used to sort a list of elements in ascending order. It removes duplicate elements if they exist in the input list. In a sentence sorting program, it arranges words alphabetically while ignoring punctuation marks if not included in sorting. This ensures proper ordering of words within the sentence.

4. Can Prolog sort words while maintaining their original case?

By default, Prolog sorts words in a case-sensitive manner, meaning uppercase letters come before lowercase ones. To achieve case-insensitive sorting, words can be converted to lowercase before sorting. After sorting, the original cases can be restored using additional mapping functions. This ensures meaningful sorting in natural language processing.

5. How can this Prolog sorting program be modified to handle punctuation?

To handle punctuation, special characters should be separated from words before sorting. This can be done using preprocessing with punctuation removal or treating punctuation as separate tokens. Characters ensure that after sorting, the meaning and readability of the sentence remain intact.

Experiment 7

Aim - Write a program to implement Water Jug Problem

Theory –

The Water Jug Problem is a classic state-space search problem in AI. States are represented as (Amount in Jug1, Amount in Jug2). Goal is reaching target Z in either jug.

State Representation: (A,B) where $0 \leq A \leq X$, $0 \leq B \leq Y$

Successor Function (next_state): Generates new states via 6 operations:

- Fill Jug1: (A,B) $\rightarrow$ (X,B)
- Fill Jug2: (A,B) $\rightarrow$ (A,Y)
- Empty Jug1: (A,B) $\rightarrow$ (0,B)
- Empty Jug2: (A,B) $\rightarrow$ (A,0)
- Pour Jug1 $\rightarrow$ Jug2: Transfer $\min(A, Y-B)$
- Pour Jug2 $\rightarrow$ Jug1: Transfer $\min(B, X-A)$

Search Strategy: Breadth-First Search (BFS) using queue, ensures shortest path (minimum steps).

Procedure –

Step 1: Create File

Create exp7.pl in text editor.

Step 2: Auto-Run Setup

Add :- initialization(main). directive.

Step 3: Define BFS Search

Implement bfs/4 with state-path pairs to track solution path.

Step 4: Goal Check

Define goal/2 for target Z in either jug.

Step 5: Successor Generation

Implement 6 next_state/4 clauses for all operations.

Step 6: Path Reconstruction

Use reverse/2 and print_steps/1 to display solution.

Step 7: Interactive Main

Prompt user for X, Y, Z capacities and run solver.

Code –

```
:- initialization(main).

% Solve jug problem
solve_jug_problem(X, Y, Z) :-
    bfs([((0,0), [(0,0)]), X, Y, Z).

% BFS Search
bfs([((A,B), Path)|_], _, _, Z) :-
    goal((A,B), Z),
    write('Solution Found:'), nl,
    print_solution(Path).

bfs([((A,B), Path)|Rest], X, Y, Z) :-
    findall((Next, [Next|Path]),
        ( next_state((A,B), X, Y, Next),
          \+ member(Next, Path)
        ),
        Children),
    append(Rest, Children, NewQueue),
    bfs(NewQueue, X, Y, Z).

% Goal State
goal((Z,_), Z).
goal((_,Z), Z).

% Possible Moves
next_state((A,B), X, _, (X,B)) :- A < X.           % Fill Jug1
next_state((A,B), _, Y, (A,Y)) :- B < Y.           % Fill Jug2
next_state((A,B), _, _, (0,B)) :- A > 0.           % Empty Jug1
next_state((A,B), _, _, (A,0)) :- B > 0.           % Empty Jug2

% Pour Jug1 -> Jug2
next_state((A,B), X, Y, (A1,B1)) :-
    A > 0, B < Y,
    Transfer is min(A, Y-B),
    A1 is A - Transfer,
    B1 is B + Transfer.

% Pour Jug2 -> Jug1
```

```

next_state((A,B), X, Y, (A1,B1)) :-
    B > 0, A < X,
    Transfer is min(B, X-A),
    B1 is B - Transfer,
    A1 is A + Transfer.

% Print Solution
print_solution(Path) :-
    reverse(Path, CorrectPath),
    print_steps(CorrectPath).

print_steps([]).
print_steps([(A,B)|T]) :-
    write('('), write(A), write(','), write(B), write(')'), nl,
    print_steps(T).

% Main
main :-
    write('Enter Jug1 Capacity: '), read(X),
    write('Enter Jug2 Capacity: '), read(Y),
    write('Enter Target Amount: '), read(Z),
    solve_jug_problem(X,Y,Z).

```

Output –

```

?- [exp7].
true.

?- main.
Enter Jug1 Capacity: 5
Enter Jug2 Capacity: 3
Enter Target Amount: 4
Solution Found:
(0,0)
(5,0)
(2,3)
(2,0)
(0,3)
(3,0)
(3,3)
(5,0)
(2,3)
(4,1)
true.

?- main.
Enter Jug1 Capacity: 3
Enter Jug2 Capacity: 5
Enter Target Amount: 4
% No output - terminates naturally (impossible case)
true.

```

Viva Voce - 7

1. What is the Water Jug Problem, and why is it important?

The Water Jug Problem is a puzzle where two jugs of different capacities are used to measure an exact amount of water. It is important in computer science as it demonstrates state-space search, graph traversal algorithms (BFS/DFS), and constraint satisfaction problems. It helps in problem-solving, AI, and optimization techniques.

2. How can the Water Jug Problem be solved?

The problem can be solved using:

Brute Force - Randomly try different actions.

Graph-Based Search (BFS/DFS) - Treat jug states as nodes and find the shortest path.

Mathematical Approach - Use GCD to check if solution exists.

Dynamic Programming - Optimize state transitions to reduce redundant steps.

3. What are some real-world applications of the Water Jug Problem?

Water distribution - Optimizing resource allocation.

Robotics - Handling liquid measurement tasks.

AI and game design - Using state-space search for decision-making.

Network load balancing - Optimizing resource usage in cloud computing.

Cryptography - Using GCD-based logic in encryption algorithms.

4. What are some variations of the Water Jug Problem?

Three-Jug Problem - Three jugs instead of two.

Limited Operations - Restrictions like not emptying water.

Finite Water Supply - Constraints on available water.

Time-Based Variations - Solving the problem within a time limit.

Multi-Agent Systems - Multiple entities working together to solve the problem.

Experiment 8

Aim - Write a program to implement Hangman game using python

Theory –

Hangman is a classic word-guessing game where Player 1 secretly chooses a word and Player 2 guesses letters one by one. Correct guesses reveal letters in positions; incorrect guesses reduce limited chances (typically 6). Game ends in win (full word guessed) or loss (chances exhausted).

Key Components:

- **Secret Word:** Input by Player 1, hidden from Player 2
- **Display State:** Underscores _ for unguessed letters
- **Guessed Letters:** Track previous guesses to prevent repeats
- **Chances:** Fixed limit (6), decrements on wrong guesses
- **Win/Loss:** Full reveal vs. game over with word disclosure

Python Features Used:

- Lists for mutable word state
- input() for interactive gameplay
- os.system('cls/clear') for screen clearing
- String methods: .lower(), .isalpha()
- List operations: indexing, membership testing

Procedure –

Step 1: Create Python File

Create exp8.py in Python editor/IDLE/VS Code.

Step 2: Import Modules

Import os for platform-specific screen clearing.

Step 3: Main Game Function

Define hangman(): get secret word, clear screen, initialize state, run game loop.

Step 4: Helper Functions

- print_instructions(): Display rules for Player 2
- game_loop(): Main gameplay with win/loss checks
- display_state(): Show current word, chances, guesses
- get_guess(): Validate single alphabetic input

- `update_state()`: Reveal correct letters in positions
- `fully_guessed()`: Check if all letters revealed

Step 5: Game Logic

Loop until win/loss: display state → get guess → process → update → repeat.

Step 6: Run Program

Execute `python exp8.py` in terminal.

Code –

```
import os
def hangman():
    print("Welcome to Hangman!")

    # Player 1 enters the secret word
    secret_word = input("Player 1, enter the secret word:
").lower()

    # Clear the screen to hide the word
    os.system('cls' if os.name == 'nt' else 'clear')

    # Print instructions for Player 2
    print_instructions()

    # Initialize game state
    word = list(secret_word)
    state = ['_'] * len(word)
    chances = 6
    guessed = []

    print(f"Player 2, the word has {len(word)} letters.")
    print("You have 6 chances to guess the word.")
    print("Guess one letter at a time.")

    # Start the game loop
    game_loop(state, word, chances, guessed)

def print_instructions():
    print("\nInstructions for Player 2:")
    print("1. Guess the secret word chosen by Player 1.")
    print("2. Enter one letter at a time.")
```

```

print("3. Correct guesses reveal the letter.")
print("4. Wrong guesses reduce your chances.")
print("5. You have only 6 chances.")
print("6. Guess the full word before chances run out to win.")
print("Good luck!\n")

def game_loop(state, word, chances, guessed):
    while True:
        # Check win condition
        if fully_guessed(state):
            print("\nCongratulations! You guessed the word:", "
".join(state))
            break

        # Check lose condition
        if chances == 0:
            print("\nGame Over! You ran out of chances.")
            print("The word was:", "".join(word))
            break

        # Display current progress
        display_state(state, chances, guessed)

        # Get guess
        guess = get_guess()

        # Process guess
        if guess in guessed:
            print("You already guessed that letter.")
        elif guess in word:
            update_state(word, state, guess)
            guessed.append(guess)
            print("Correct guess!")
        else:
            chances -= 1
            guessed.append(guess)
            print("Wrong guess!")

        print("-----")

def fully_guessed(state):
    return '_' not in state

```

```

def display_state(state, chances, guessed):
    print("\nCurrent word:", " ".join(state))
    print("Chances left:", chances)
    print("Guessed letters:", guessed)

def get_guess():
    while True:
        guess = input("Guess a letter: ").lower()
        if len(guess) == 1 and guess.isalpha():
            return guess
        print("Invalid input! Please enter a single letter.")

def update_state(word, state, letter):
    for i in range(len(word)):
        if word[i] == letter:
            state[i] = letter

# Start the game
if __name__ == "__main__":
    hangman()

```

Output –

```

$ python exp8.py
Welcome to Hangman!
Player 1, enter the secret word: python

[Screen clears]

Instructions for Player 2:
1. Guess the secret word chosen by Player 1.
[...]

Player 2, the word has 6 letters.
You have 6 chances to guess the word.
Guess one letter at a time.

Current word: _ _ _ _ _
Chances left: 6
Guessed letters: []
Guess a letter: p
Correct guess!
-----

```

Viva Voce - 8

1. What is the basic logic behind the Hangman game in Python?

The Hangman game works by letting a player guess letter of a hidden word. For each correct guess, the letter will be revealed in its correct positions. For wrong guesses, the player loses one of the limited number of chances. The game continues until the player either guesses the full word correctly or runs out of attempts.

2. How do you handle user inputs and check if the guessed letter is correct?

User inputs are taken using the `input()` function. The guessed letter is checked using a simple condition `if letter in word`. If it's present, we reveal it in the appropriate places in the word display; otherwise, we reduce the remaining attempts. We also store guessed letters to avoid repetition.

3. How do you display the progress of the guessed word during the game?

We use a list of underscores e.g., `['_ _ _']`, representing the hidden word. As the player guesses correct letters, we update the list by replacing the underscores with the correct letters at their corresponding positions. The list is printed each round to show progress.

4. How do you keep track of the number of chances left for the player?

We initialize a variable, e.g., `chances = 6`, and decrement it each time the player makes an incorrect guess. The game ends when the variable reaches zero. We display the remaining chances after each guess to keep the player informed.

5. Can you explain how to detect a win or loss condition in the game?

A win is detected when the list of guessed letters matches the full word, meaning all letters have been guessed correctly. A loss is detected when the number of chances becomes zero before guessing the full word. Both conditions trigger end messages accordingly.

Experiment 9

Aim - Write a program to implement Hangman game using prolog

Theory –

Hangman is a two-player word guessing game implemented using Prolog's logic programming paradigm. Player 1 enters secret word; Player 2 guesses letters interactively. Prolog handles state management, input validation, and game logic through recursive predicates.

Prolog Implementation Strategy:

- **State:** List of chars (['_','_','_']) representing revealed word
- **Word:** List of lowercase secret word letters
- **Chances:** Integer counter (starts at 6)
- **Guessed:** List of previously guessed letters
- **Recursion:** game_loop/4 calls itself with updated state

Key Prolog Features:

- read/1: Interactive input
- atom_chars/2: String ↔ list conversion
- downcase_atom/2: Case normalization
- maplist/3: Parallel list transformation
- Pattern matching & unification for logic
- Backtracking prevention via explicit state passing

Procedure –

Step 1: Create Prolog File

Create exp9.pl.

Step 2: Auto-Start

`:- initialization(hangman).` runs game on load.

Step 3: Main Predicate

`hangman/0:` Get secret word, print instructions, initialize state, start loop.

Step 4: Game Loop Logic

`game_loop(State, Word, Chances, Guessed):` Check win/loss, display, get guess, recurse.

Step 5: Helper Predicates

- `fully_guessed/1:` No '_' in state

- display_state/3: Format current game info
- get_guess/1: Read/validate single letter
- update_state/4: Reveal correct positions

Step 6: Load & Run

swipl exp9.pl (auto-starts).

Code –

```
:- initialization(hangman).

% Main predicate
hangman :-
    write('Welcome to Hangman!'), nl,
    write('Player 1, enter the secret word followed by a period
(e.g., hello.): '),
    read(SecretAtom),
    downcase_atom(SecretAtom, LowerSecret),
    atom_chars(LowerSecret, Word),

    print_instructions,

    length(Word, Len),
    format('Player 2, the word has ~w letters.~n', [Len]),
    write('You have 6 chances to guess the word. '), nl,
    write('Guess one letter at a time. '), nl,

    length(State, Len),
    maplist(= '(_)', State),

    game_loop(State, Word, 6, []).

% Instructions
print_instructions :-
    nl,
    write('Instructions: '), nl,
    write('Guess one letter at a time. '), nl,
    write('Correct guess reveals positions. '), nl,
    write('Wrong guess loses a chance. '), nl,
    write('You have only 6 chances. '), nl, nl.
```

```

% Game Loop
game_loop(State, Word, _, _) :-
    fully_guessed(State),
    atomic_list_concat(State, '', WordAtom),
    format('Congratulations! You guessed the word: ~w~n',
[WordAtom])).

game_loop(_, Word, 0, _) :-
    atomic_list_concat(Word, '', WordAtom),
    format('Game Over! The word was: ~w~n', [WordAtom])).

game_loop(State, Word, Chances, Guessed) :-
    display_state(State, Chances, Guessed),
    get_guess(Guess),

    ( member(Guess, Guessed) ->
        write('You already guessed that letter. '), nl,
        game_loop(State, Word, Chances, Guessed)

    ; member(Guess, Word) ->
        update_state(Word, State, Guess, NewState),
        write('Correct guess! '), nl,
        game_loop(NewState, Word, Chances, [Guess|Guessed])

    ; NewChances is Chances - 1,
        write('Wrong guess! '), nl,
        game_loop(State, Word, NewChances, [Guess|Guessed])
    ).

% Check win
fully_guessed(State) :-
    \+ member('_', State).

% Display game state
display_state(State, Chances, Guessed) :-
    atomic_list_concat(State, ' ', StateAtom),
    format('Word: ~w~n', [StateAtom]),
    format('Chances left: ~w~n', [Chances]),
    format('Guessed letters: ~w~n~n', [Guessed])).

% Get guess
get_guess(Guess) :-

```

```

write('Guess a letter: '),
read(GuessAtom),
downcase_atom(GuessAtom, Lower),
atom_chars(Lower, [Guess]).

% Update state
update_state(Word, OldState, Letter, NewState) :-
    maplist(update_letter(Letter), Word, OldState, NewState).

update_letter(Letter, Letter, _, Letter).
update_letter(Letter, Actual, Old, Old) :-
    Letter \= Actual.

```

Output –

```

?- [exp9].
true.

Welcome to Hangman!
Player 1, enter the secret word followed by a period (e.g., hello.): prolog.

Instructions:
[...]

Player 2, the word has 6 letters.
You have 6 chances to guess the word.
Guess one letter at a time.

Word: _ _ _ _ _
Chances left: 6
Guessed letters: []

Guess a letter: p
Correct guess!
Word: p _ _ _ _
Chances left: 6
[...]

```


Viva Voce - 9

Q. How does Hangman work in Prolog by recursively processing guesses?

Prolog works by recursively processing Hangman guesses, updating the guessed word, and checking if the word has been fully guessed or if the player has exhausted all chances. It uses facts and rules to store incorrect guesses, letters, and remaining attempts.

Q. How does Prolog handle user input and update the game state dynamically?

Prolog uses `read/1` and updates the game state using recursive rules. Each turn, it checks the guessed letter in the word, and then checks the guessed letter or ends with the updated attempts displayed.

Q. What Prolog data structures are used to store the game state?

(e.g., lists). Counters for secret word, guessed letters, and a current state of the word are maintained as numeric values that decrement on incorrect guesses. Facts and recursion help modify these dynamically.

Q. How does Prolog determine if the game is won or lost?

By comparing when all letters in the secret word have been correctly guessed. The game is lost when the remaining attempts reach zero, at which point the program displays the correct word and ends.

Q. Why is Prolog suitable for implementing Hangman?

Prolog is suitable because it efficiently handles pattern matching and recursion, which are key to checking guesses, updating the word state, and controlling a good game flow for text-based games like Hangman.

Experiment 10

Aim - Write a program to implement Tic-Tac-Toe game using Prolog

Theory –

Tic-Tac-Toe is a classic two-player strategy game implemented using Prolog's logic programming paradigm. Players X and O alternately mark positions on a 3x3 grid. The game ends when one player gets three marks in a row, column, or diagonal, or when all cells are filled resulting in a draw. Prolog handles game state, move validation, and win/draw detection through pattern matching and recursive predicates.

Procedure –

Step 1: Create Prolog File

Create exp10.pl and open it in your preferred editor.

Step 2: Define Initial Board

initial_board/1 sets a 9-element list of empty atoms as the starting state.

Step 3: Start the Game

Call start/0 which prints instructions and board layout, then invokes play/0.

Step 4: Game Loop

play_game/2 prints the current board, checks for win/draw, else prompts the current player for a move.

Step 5: Move Validation

make_move/3 reads position 1-9, validates via valid_move/2 (in-range and empty cell), replaces with player symbol using replace/4.

Step 6: Win and Draw Detection

winner/2 checks row_win, col_win, diag_win via pattern matching. full_board/1 detects draw when no empty cell remains.

Step 7: Load and Run

swipl exp10.pl, then call start. to begin the game interactively.

Code –

```
% Define the initial empty board
```

```
initial_board([empty, empty, empty, empty, empty, empty, empty, empty, empty, empty]).
```

```
% Define the game loop
```

```
play :-
```

```
    initial_board(Board),
```

```
    play_game(Board, x).
```

```
play_game(Board, Player) :-
```

```
    print_board(Board),
```

```
    (    winner(Board, x) -> % Check if X has won
```

```
        write('Player X wins!'), nl
```

```
    ;    winner(Board, o) -> % Check if O has won
```

```
        write('Player O wins!'), nl
```

```
    ;    full_board(Board) -> % Check if the board is full
```

```
        write('It\'s a draw!'), nl
```

```
    ;    make_move(Board, Player, NewBoard), % Player makes a move
```

```
        next_player(Player, NextPlayer), % Switch to the next
```

```
player
```

```
    play_game(NewBoard, NextPlayer)
```

```
).
```

```
% Print the board
```

```
print_board([A, B, C, D, E, F, G, H, I]) :-
```

```
    format('~w | ~w | ~w~n', [A, B, C]),
```

```
    write('-----'), nl,
```

```
    format('~w | ~w | ~w~n', [D, E, F]),
```

```
    write('-----'), nl,
```

```
    format('~w | ~w | ~w~n', [G, H, I]).
```

```
% Check if a player has won
```

```
winner(Board, Player) :-
```

```
    (    row_win(Board, Player)
```

```
    ;    col_win(Board, Player)
```

```
    ;    diag_win(Board, Player)
```

```
).
```

```
% Check for a winning row
```

```
row_win([A, B, C, _, _, _, _, _, _], Player) :-
```

```
    A = Player, B = Player, C = Player.
```

```
row_win([_, _, _, D, E, F, _, _, _], Player) :-
```

```
    D = Player, E = Player, F = Player.
```

```
row_win([_, _, _, _, _, _, G, H, I], Player) :-
```

```

    G = Player, H = Player, I = Player.

% Check for a winning column
col_win([A, _, _, D, _, _, G, _, _], Player) :-
    A = Player, D = Player, G = Player.
col_win([_, B, _, _, E, _, _, H, _], Player) :-
    B = Player, E = Player, H = Player.
col_win([_, _, C, _, _, F, _, _, I], Player) :-
    C = Player, F = Player, I = Player.

% Check for a winning diagonal
diag_win([A, _, _, _, E, _, _, _, I], Player) :-
    A = Player, E = Player, I = Player.
diag_win([_, _, C, _, E, _, G, _, _], Player) :-
    C = Player, E = Player, G = Player.

% Check if the board is full
full_board(Board) :-
    \+ member(empty, Board).

% Make a move for the current player
make_move(Board, Player, NewBoard) :-
    write('Enter a position (1-9): '),
    read(Pos),
    (    valid_move(Board, Pos)
    ->  replace(Pos, Player, Board, NewBoard)
    ;   write('Invalid move! Position must be between 1-9 and
empty. '), nl,
        make_move(Board, Player, NewBoard) % Ask again for valid
move
    ).

% Validate the move (position must be between 1 and 9 and the cell
must be empty)
valid_move(Board, Pos) :-
    integer(Pos),
    Pos >= 1,
    Pos <= 9,
    nth1(Pos, Board, Cell),
    Cell = empty.

% Replace a position on the board with the current player's symbol

```

```

replace(Pos, Player, Board, NewBoard) :-
    nth1(Pos, Board, _, TempBoard),
    nth1(Pos, NewBoard, Player, TempBoard).

% Switch to the next player
next_player(x, o).
next_player(o, x).

% Helper predicate to start the game with proper initialization
start :-
    write('Welcome to Tic-Tac-Toe!'), nl,
    write('Players take turns entering positions (1-9).'), nl,
    write('Board positions are:'), nl,
    write('1 | 2 | 3'), nl,
    write('-----'), nl,
    write('4 | 5 | 6'), nl,
    write('-----'), nl,
    write('7 | 8 | 9'), nl, nl,
    play.

```

Output –

```

Welcome to Tic-Tac-Toe!
Players take turns entering positions (1-9).
Board positions are:
1 | 2 | 3
-----
4 | 5 | 6
-----
7 | 8 | 9

empty | empty | empty
-----
empty | empty | empty
-----
empty | empty | empty

Enter a position (1-9): 5.
empty | empty | empty
-----
empty | x | empty
-----
empty | empty | empty

Enter a position (1-9): 1.
o | empty | empty
-----
empty | x | empty
-----
empty | empty | empty

Enter a position (1-9): 9.
o | empty | empty
-----
empty | x | empty
-----
empty | empty | x

Enter a position (1-9): 3.

```

```

empty | x | empty
-----
empty | empty | x

Enter a position (1-9): 7
o | empty | o
-----
empty | x | empty
-----
o | empty | x

Enter a position (1-9): 2
o | x | o
-----
empty | x | empty
-----
o | empty | x

Enter a position (1-9): 8
o | x | o
-----
empty | x | o
-----
o | empty | x

Enter a position (1-9): 4
o | x | o
-----
x | x | o
-----
o | empty | x

Enter a position (1-9): 6
o | x | o
-----
x | x | o
-----
o | o | x
It's a draw!

```

Viva Voce - 10

Q. How does Tic-Tac-Toe work in Prolog using recursive game loop?

Prolog implements the game loop via `play_game/2` which recursively calls itself after each move, passing the updated board and the next player. It checks win or draw conditions before requesting the next move, ensuring the game progresses until a terminal state is reached.

Q. How does Prolog represent and update the Tic-Tac-Toe board?

The board is a 9-element Prolog list where each element is empty, x, or o. The predicate `nth1/4` is used to replace an element at a given 1-based index with the current player's symbol, producing a new board list while leaving the original unchanged.

Q. What Prolog predicates are used to check for a win in Tic-Tac-Toe?

Three predicates handle win detection: `row_win/2` checks all three rows, `col_win/2` checks all three columns, and `diag_win/2` checks both diagonals. Each uses pattern matching on the board list to verify that all three relevant positions hold the same player symbol.

Q. How does Prolog validate player input in this game?

The `valid_move/2` predicate checks that the input is an integer in the range 1–9 and that the corresponding cell in the board list is still empty using `nth1/3`. If invalid, `make_move/3` recursively prompts the player again until a valid position is entered.

Q. Why is Prolog suitable for implementing Tic-Tac-Toe?

Prolog is suitable because its pattern matching naturally expresses win conditions as declarative rules, and its list predicates elegantly model the board. Recursion replaces loops, and unification simplifies state transitions, making game logic concise and readable.

Experiment 11

Aim - Write a program to remove stop words for a given passage from a text file using NLTK

Theory –

Stop word removal is a fundamental text preprocessing step in Natural Language Processing (NLP). Stop words are common words such as "is", "the", "and", "in" that appear frequently in text but carry little meaningful information for analysis tasks like classification or information retrieval. By removing them, we reduce noise and improve the efficiency of downstream NLP pipelines.

Processing Pipeline:

- **Step 1:** Read raw text from input.txt file.
- **Step 2:** Remove punctuation using str.maketrans and convert text to lowercase.
- **Step 3:** Split text into individual tokens (words) using split().
- **Step 4:** Filter out tokens that appear in the NLTK stopwords set.
- **Step 5:** Join remaining meaningful words and write to output.txt.

Procedure –

Step 1: Install Required Library

Ensure NLTK is installed: `pip install nltk`. The stopwords corpus is downloaded automatically via `nltk.download('stopwords', quiet=True)` at runtime.

Step 2: Create Input File

Create a text file named input.txt in the same directory and write or paste the passage whose stop words are to be removed.

Step 3: Import Modules

Import `nltk`, `string`, and `nltk.corpus.stopwords`. Download the stopwords corpus if not already present on the system.

Step 4: Define remove_stop_words Function

The function reads the input file, strips punctuation, lowercases text, splits into tokens, filters stop words using set lookup, joins the result, and writes it to the output file.

Step 5: Define main Function

The `main()` function sets `input_file = './input.txt'` and `output_file = './output.txt'`, then calls `remove_stop_words` with these paths.

Step 6: Run the Program

Execute python exp11.py from the terminal. The filtered passage will be saved to output.txt and a confirmation message will be printed.

Step 7: Verify Output

Open output.txt to verify that stop words have been removed and only meaningful content words remain in the passage.

Code –

```
import nltk
import string

nltk.download('stopwords', quiet=True)
from nltk.corpus import stopwords

def remove_stop_words(input_file, output_file):
    stop_words = set(stopwords.words('english'))

    try:
        with open(input_file, 'r', encoding='utf-8') as file:
            text = file.read()

            text = text.translate(str.maketrans('', '',
string.punctuation)).lower()
            words = text.split()
            filtered_words = [word for word in words if word not in
stop_words]
            filtered_text = ' '.join(filtered_words)

            with open(output_file, 'w', encoding='utf-8') as file:
                file.write(filtered_text)

            print(f'Stop words removed. Result written to
{output_file}')

    except FileNotFoundError:
        print(f'Error: Input file {input_file} not found.')
    except Exception as e:
        print(f"An error occurred: {e}")

def main():
    input_file = './input.txt'
    output_file = './output.txt'
```



```
remove_stop_words(input_file, output_file)

if __name__ == "__main__":
    main()
```

Output –

```
PS C:\Users\HP\Desktop\AI EXPERIMENTS> python exp11.py
Stop words removed. Result written to ./output.txt
PS C:\Users\HP\Desktop\AI EXPERIMENTS>
```

input.txt

Artificial intelligence is the simulation of human intelligence processes by machines, especially computer systems. These processes include learning, reasoning, and self-correction. AI is used in expert systems, natural language processing, and machine vision.

output.txt

artificial intelligence simulation human intelligence processes machines especially computer systems processes include learning reasoning self correction ai used expert systems natural language processing machine vision

Viva Voce - 11

Q. What are stop words and why are they removed in NLP?

Stop words are high-frequency words like "the", "is", "and" that appear in almost every document but carry minimal semantic information. They are removed to reduce dimensionality, improve processing speed, and allow NLP models to focus on meaningful content words.

Q. How does NLTK's stopwords corpus work in this program?

NLTK provides a pre-built list of ~179 English stop words accessible via `stopwords.words('english')`. The program converts this list into a Python set for $O(1)$ average lookup time, then filters each token in the passage using a list comprehension.

Q. Why is punctuation removed before filtering stop words?

Punctuation attached to words (e.g., "word." or "word,") would prevent exact matching against the stop words set. Removing punctuation first ensures tokens like "the." are correctly identified as "the" and filtered out.

Q. What is the role of `str.maketrans` and `translate` in this program?

`str.maketrans(", ", string.punctuation)` creates a translation table that maps every punctuation character to `None`. The `translate()` method applies this table to the text string, efficiently deleting all punctuation in a single pass without loops.

Q. How would you extend this program to support multiple languages?

NLTK's stopwords corpus supports multiple languages such as French, German, Spanish, etc. The language can be passed as a parameter: `stopwords.words('french')`. The program could be extended by accepting a language argument and passing it to the `stopwords.words()` call.

Experiment 12

Aim - Write a program to implement stemming for a given sentence using NLTK

Theory –

Stemming is a text normalization technique in Natural Language Processing (NLP) that reduces inflected or derived words to their root base form, called a stem. For example, "running", "runner", and "runs" all reduce to the stem "run". Stemming improves information retrieval, text mining, and search accuracy by grouping related word forms under a single token, reducing vocabulary size and computational overhead in downstream NLP tasks.

Types of Stemmers used:

- **PorterStemmer**: The most widely used stemmer, based on a series of suffix-stripping rules. It is fast and produces reasonably accurate stems for English text.
- **SnowballStemmer**: An improved version of the Porter algorithm (also called Porter2). More accurate than PorterStemmer and supports multiple languages. Initialized with a language parameter e.g. 'english'.
- **LancasterStemmer**: The most aggressive stemmer – it applies iterative rules and produces the shortest possible stems. It can over-stem words, resulting in less readable but highly compressed output.

Procedure –

Step 1: Install NLTK

Ensure NLTK is installed via `pip install nltk`. The punkt tokenizer data is downloaded automatically at runtime using `nltk.download('punkt', quiet=True)`.

Step 2: Import Required Modules

Import `nltk`, `string`, `PorterStemmer`, `SnowballStemmer`, `LancasterStemmer` from `nltk.stem`, and `word_tokenize` from `nltk.tokenize`.

Step 3: Define SentenceStemmer Class

Create a class that initialises all three stemmer objects in `__init__`. This avoids re-instantiating them for every sentence.

Step 4: Implement pre_process_text Method

This method lowercases the sentence, strips all punctuation using `str.maketrans` and `translate`, then tokenizes it into a word list using `word_tokenize`.

Step 5: Implement stem_sentence Method

Accept a sentence and stemmer_type parameter ('porter', 'snowball', or 'lancaster'). Apply the selected stemmer to each token and return a dictionary with original words, stemmed words, and the joined stemmed sentence.

Step 6: Implement compare_stemmers Method

Iterate over all three stemmer types, call stem_sentence for each, and accumulate results into a dictionary keyed by stemmer name for easy comparison.

Step 7: Run and Verify

Execute python exp12.py. Observe how each stemmer reduces words differently, especially for irregular or complex words in the sample sentences.

Code –

```
import nltk
import string
from nltk.stem import PorterStemmer, SnowballStemmer, LancasterStemmer
from nltk.tokenize import word_tokenize
nltk.download('punkt', quiet=True)

class SentenceStemmer:
    def __init__(self):
        self.porter_stemmer = PorterStemmer()
        self.snowball_stemmer = SnowballStemmer('english')
        self.lancaster_stemmer = LancasterStemmer()

    def pre_process_text(self, sentence):
        # lowercase
        sentence = sentence.lower()
        # remove punctuation
        sentence = sentence.translate(str.maketrans("", "", string.punctuation))
        # tokenize
        return word_tokenize(sentence)

    def stem_sentence(self, sentence, stemmer_type='porter'):
        words = self.pre_process_text(sentence)
        if stemmer_type == 'porter':
            stemmer = self.porter_stemmer
        elif stemmer_type == 'snowball':
            stemmer = self.snowball_stemmer
        elif stemmer_type == 'lancaster':
            stemmer = self.lancaster_stemmer
        else:
```

```

        raise ValueError("Invalid stemmer type")
    stemmed_words = [stemmer.stem(word) for word in words]
    return {
        'original_words': words,
        'stemmed_words': stemmed_words,
        'stemmed_sentence': ' '.join(stemmed_words)
    }

def compare_stemmers(self, sentence):
    stemmer_types = ['porter', 'snowball', 'lancaster']
    results = {}
    for stemmer_type in stemmer_types:
        results[stemmer_type] = self.stem_sentence(sentence, stemmer_type)
    return results

def main():
    stemmer = SentenceStemmer()
    sentences = [
        "Running quickly through the beautiful gardens.",
        "The programmers are developing interesting software applications.",
        "I am easily understanding complex machine learning concepts."
    ]
    for sentence in sentences:
        print(f"\nOriginal Sentence: {sentence}")
        comparison = stemmer.compare_stemmers(sentence)
        for stemmer_type, result in comparison.items():
            print(f"\n{stemmer_type.capitalize()} Stemmer:")
            print(f"Original Words: {result['original_words']}")
            print(f"Stemmed Words: {result['stemmed_words']}")
            print(f"Stemmed Sentence: {result['stemmed_sentence']}")

if __name__ == "__main__":
    main()

```

Output –

Original Sentence: Running quickly through the beautiful gardens.

Porter Stemmer:

Original Words: ['running', 'quickly', 'through', 'the', 'beautiful', 'gardens']

Stemmed Words: ['run', 'quickli', 'through', 'the', 'beauti', 'garden']

Stemmed Sentence: run quickli through the beauti garden

Snowball Stemmer:

Original Words: ['running', 'quickly', 'through', 'the', 'beautiful', 'gardens']

Stemmed Words: ['run', 'quick', 'through', 'the', 'beauti', 'garden']

Stemmed Sentence: run quick through the beauti garden

Lancaster Stemmer:

Original Words: ['running', 'quickly', 'through', 'the', 'beautiful', 'gardens']

Stemmed Words: ['run', 'quick', 'through', 'the', 'beaut', 'gard']

Stemmed Sentence: run quick through the beaut gard

Original Sentence: The programmers are developing interesting software applications.

Porter Stemmer:

Original Words: ['the', 'programmers', 'are', 'developing', 'interesting', 'software', 'applications']

Stemmed Words: ['the', 'programm', 'are', 'develop', 'interest', 'softwar', 'applic']

Stemmed Sentence: the programm are develop interest softwar applic

Snowball Stemmer:

Original Words: ['the', 'programmers', 'are', 'developing', 'interesting', 'software', 'applications']

Stemmed Words: ['the', 'programm', 'are', 'develop', 'interest', 'softwar', 'applic']

Stemmed Sentence: the programm are develop interest softwar applic

Lancaster Stemmer:

Original Words: ['the', 'programmers', 'are', 'developing', 'interesting', 'software', 'applications']

Stemmed Words: ['the', 'program', 'ar', 'develop', 'interest', 'softw', 'apply']

Stemmed Sentence: the program ar develop interest softw apply

Original Sentence: I am easily understanding complex machine learning concepts.

Porter Stemmer:

Original Words: ['i', 'am', 'easily', 'understanding', 'complex', 'machine', 'learning', 'concepts']

Stemmed Words: ['i', 'am', 'easili', 'understand', 'complex', 'machin', 'learn', 'concept']

Stemmed Sentence: i am easili understand complex machin learn concept

Snowball Stemmer:

Original Words: ['i', 'am', 'easily', 'understanding', 'complex', 'machine', 'learning', 'concepts']

Stemmed Words: ['i', 'am', 'easili', 'understand', 'complex', 'machin', 'learn', 'concept']

Stemmed Sentence: i am easili understand complex machin learn concept

Lancaster Stemmer:

Original Words: ['i', 'am', 'easily', 'understanding', 'complex', 'machine', 'learning', 'concepts']

Stemmed Words: ['i', 'am', 'easy', 'understand', 'complex', 'machin', 'learn', 'concept']

Stemmed Sentence: i am easy understand complex machin learn concept

Viva Voce - 12

Q. What is stemming and how does it differ from lemmatization?

Stemming reduces words to their root form by chopping off suffixes using heuristic rules, which may not always produce a real dictionary word (e.g., "studies" → "studi"). Lemmatization, on the other hand, uses vocabulary and morphological analysis to return the actual base form (e.g., "studies" → "study"). Stemming is faster but less accurate; lemmatization is slower but linguistically correct.

Q. How do PorterStemmer, SnowballStemmer, and LancasterStemmer differ?

PorterStemmer applies a fixed set of suffix-stripping rules and is moderate in aggressiveness. SnowballStemmer (Porter2) is an improved, slightly more accurate version that also supports multiple languages. LancasterStemmer is the most aggressive, applying iterative rules to produce very short stems, which can sometimes over-stem and reduce words to unrecognisable forms.

Q. Why is pre-processing (lowercasing and punctuation removal) done before stemming?

Stemmers are case-sensitive and punctuation-aware. Without lowercasing, "Running" and "running" would be treated as different tokens. Punctuation like periods or commas attached to words would prevent correct suffix matching. Pre-processing ensures all tokens are in a clean, uniform form before the stemming algorithm is applied.

Q. What does word_tokenize do and why is it preferred over split()?

word_tokenize uses a trained Punkt tokenizer that correctly handles contractions (e.g., "don't" → ["do", "n't"]), hyphenated words, and edge cases that a simple split() would miss. It produces linguistically accurate token boundaries, which leads to better stemming results compared to naive whitespace splitting.

Q. What are the practical applications of stemming in NLP?

Stemming is widely used in search engines to match query terms with document terms that share the same root (e.g., searching "run" also returns results for "running" and "runner"). It is also used in text classification, sentiment analysis, information retrieval, and document clustering to reduce vocabulary size and normalise features for machine learning models.

Experiment 13

Aim - Write a program to POS (Part of Speech) tagging for the given sentence using NLTK

Theory – Part of Speech (POS) tagging is the process of labelling each word in a sentence with its corresponding grammatical category, such as noun, verb, adjective, adverb, pronoun, and so on. POS tagging is a fundamental step in NLP pipelines and serves as a foundation for tasks like named entity recognition, parsing, sentiment analysis, and machine translation. NLTK provides a pre-trained Averaged Perceptron Tagger that assigns Penn Treebank POS tags to tokenised words.

Common Penn Treebank POS Tags used in this program:

- NN / NNS / NNP: Noun singular / Noun plural / Proper noun singular
- VB / VBD / VBG: Verb base form / Verb past tense / Verb gerund or present participle
- JJ: Adjective
- RB: Adverb
- DT: Determiner (the, a, an)
- IN: Preposition or subordinating conjunction
- PRP: Personal pronoun (I, he, she, they)
- CC: Coordinating conjunction (and, but, or)

Procedure –

Step 1: Install Required Libraries

Ensure NLTK and pandas are installed: `pip install nltk pandas`. NLTK corpora punkt and averaged_perceptron_tagger are downloaded automatically at runtime.

Step 2: Import Modules

Import `nltk`, `word_tokenize` from `nltk`, and `pandas` as `pd`. Download the punkt and averaged_perceptron_tagger datasets using `nltk.download()` with `quiet=True`.

Step 3: Define pos_tag_sentence Function

The function accepts a sentence string, tokenizes it using `word_tokenize`, and passes the token list to `nltk.pos_tag()` to obtain (word, tag) tuples.

Step 4: Display Results as DataFrame

Convert the list of (word, tag) tuples into a pandas DataFrame with columns ['Word', 'POS Tag'] and print it for a clean tabular view.

Step 5: Print Tag Explanations

Build a `common_tags` dictionary mapping Penn Treebank tag codes to human-readable descriptions. Extract unique tags from the sentence output and print their explanations.

Step 6: Define Main Block

In the if `__name__ == "__main__"` block, define a list of three sample sentences, iterate over them calling `pos_tag_sentence()`, and print a separator line between results using `"*50"`.

Step 7: Run and Observe

Execute `python exp13.py`. For each sentence, observe the tokenized words, their assigned POS tags in the DataFrame, and the explanations of the unique tags found.

Code –

```
import nltk
from nltk import word_tokenize
import pandas as pd
# Download necessary data
nltk.download('punkt', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)

def pos_tag_sentence(sentence):
    # Tokenize
    words = word_tokenize(sentence)
    # POS tagging
    pos_tags = nltk.pos_tag(words)
    # DataFrame
    df = pd.DataFrame(pos_tags, columns=['Word', 'POS Tag'])
    print(f"Original Sentence: {sentence}\n")
    print("POS Tagging Results:")
    print(df)
    # Common tags
    common_tags = {
        'NN': 'Noun, singular',
        'NNS': 'Noun, plural',
        'NNP': 'Proper noun, singular',
        'VB': 'Verb, base form',
        'VBD': 'Verb, past tense',
        'VBG': 'Verb, gerund',
        'JJ': 'Adjective',
        'RB': 'Adverb',
        'DT': 'Determiner',
        'IN': 'Preposition',
        'PRP': 'Pronoun',
        'CC': 'Conjunction'
    }
    print("\nCommon POS Tag Explanations:")
```

```

unique_tags = set([tag for _, tag in pos_tags])
for tag in unique_tags:
    if tag in common_tags:
        print(f"{tag}: {common_tags[tag]}")
return pos_tags

if __name__ == "__main__":
    sentences = [
        "NLTK is a powerful Python library for natural language
processing."
    ]
    for i, sentence in enumerate(sentences):
        pos_tag_sentence(sentence)
        if i < len(sentences) - 1:
            print("\n" + "="*50 + "\n")

```

Output –

Original Sentence: NLTK is a powerful Python library for natural language processing.

POS Tagging Results:

	Word	POS	Tag
0	NLTK	NNP	
1	is	VBZ	
2	a	DT	
3	powerful	JJ	
4	Python	NNP	
5	library	NN	
6	for	IN	
7	natural	JJ	
8	language	NN	
9	processing	VBG	
10	.	.	

Common POS Tag Explanations:

NNP: Proper noun, singular
 VBZ: Verb, 3rd person singular present
 DT: Determiner
 JJ: Adjective
 NN: Noun, singular
 IN: Preposition
 VBG: Verb, gerund

Viva Voce – 13

Q. What is POS tagging and why is it important in NLP?

POS tagging assigns a grammatical category (noun, verb, adjective, etc.) to each word in a sentence. It is important because it provides structural information about text that downstream tasks such as named entity recognition, syntactic parsing, sentiment analysis, and machine translation rely on to understand the role and meaning of each word in context.

Q. How does NLTK's Averaged Perceptron Tagger work?

The Averaged Perceptron Tagger is a pre-trained statistical model that assigns POS tags by looking at features of each word and its surrounding context, such as prefix, suffix, capitalisation, and neighbouring words.

Q. What is the Penn Treebank tagset used by NLTK's `pos_tag`?

Penn Treebank is a widely used tagset with around 36 tags. Common tags include NN (noun singular), VB (verb base form), JJ (adjective), RB (adverb), DT (determiner), IN (preposition), PRP (personal pronoun), and CC (coordinating conjunction). NLTK's `nlk.pos_tag()` function returns tags from this tagset by default.

Q. Why is pandas DataFrame used to display POS tag results?

The `pos_tag()` function returns a list of (word, tag) tuples which can be difficult to read in raw Python format. Converting it to a pandas DataFrame with labelled columns ['Word', 'POS Tag'] produces a clean, aligned tabular display that is much easier to interpret, especially when processing longer sentences with many tokens.

Q. What is the difference between POS tagging and Named Entity Recognition (NER)?

POS tagging assigns grammatical categories (noun, verb, adjective, etc.) to every word in a sentence. Named Entity Recognition (NER) identifies and classifies specific real-world entities such as person names, organisations, locations, and dates.

Experiment 14

Aim - Write a program to implement Lemmatization using NLTK

Theory –

Lemmatization is a text normalization technique in Natural Language Processing (NLP) that reduces a word to its base dictionary form, known as a lemma. Unlike stemming, which mechanically strips suffixes, lemmatization uses vocabulary and morphological analysis to return a linguistically valid word. For example, "running" → "run", "better" → "good", "studies" → "study". NLTK's WordNetLemmatizer leverages the WordNet lexical database to look up the correct lemma for each word based on its part of speech.

Code –

```
import nltk
from nltk.stem import WordNetLemmatizer
from nltk import word_tokenize, pos_tag
import pandas as pd
# Download required data
nltk.download('punkt', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)

# Map POS tag to WordNet POS
def get_wordnet_pos(tag):
    tag_dict = {
        'J': nltk.corpus.wordnet.ADJ,
        'N': nltk.corpus.wordnet.NOUN,
        'V': nltk.corpus.wordnet.VERB,
        'R': nltk.corpus.wordnet.ADV
    }
    return tag_dict.get(tag[0], nltk.corpus.wordnet.NOUN)

def lemmatize_text(text):
    lemmatizer = WordNetLemmatizer()
    tokens = word_tokenize(text)
    pos_tags = pos_tag(tokens)
    lemmatized_words = []
    for word, tag in pos_tags:
        wordnet_pos = get_wordnet_pos(tag)
        lemma = lemmatizer.lemmatize(word, wordnet_pos)
        lemmatized_words.append((word, tag, lemma))
    df = pd.DataFrame(lemmatized_words,
```

```

        columns=['Original Word', 'POS Tag',
'Lemmaized Word'])
    print(f"Original Text: {text}\n")
    print("Lemmatization Results:")
    print(df)
    print()
    lemmaized_text = ' '.join([word for _, _, word in
lemmaized_words])
    print(f"Lemmaized Text: {lemmaized_text}")
    return lemmaized_words

if __name__ == "__main__":
    print("="*60)
    print("NLTK LEMMATIZATION DEMONSTRATION")
    print("="*60 + "\n")
    examples = [
        "The cats are running and jumping in the gardens",
        "I am studying studies studied by the best students"
    ]
    for i, example in enumerate(examples, 1):
        print(f"Example {i}:")
        lemmaize_text(example)
        if i < len(examples):
            print("\n" + "-"*60 + "\n")

```

Output –

```

Original Text: The cats are running and jumping in the gardens

Lemmatization Results:
cats → cat
running → run
jumping → jump
gardens → garden

Lemmaized Text:
The cat be run and jump in the garden


Original Text: I am studying studies studied by the best students

study, studies, studied → study
students → student

Lemmaized Text:
I be study study study by the good student

```

Viva Voce - 14

Q. What is lemmatization and how does it differ from stemming?

Lemmatization reduces a word to its base dictionary form (lemma) using vocabulary and morphological analysis, always producing a valid word (e.g., "better" → "good"). Stemming uses heuristic suffix-stripping rules that are faster but may produce non-words (e.g., "studies" → "studi"). Lemmatization is more accurate; stemming is faster.

Q. Why is POS tagging important before lemmatization in this program?

The correct lemma of a word depends on its grammatical role. For example, "running" as a verb lemmatizes to "run", but as a noun it stays "running". Without POS context, WordNetLemmatizer defaults to treating every word as a noun, leading to incorrect lemmas for verbs and adjectives. POS-aware lemmatization improves accuracy significantly.

Q. How does `get_wordnet_pos` work and why is it needed?

NLTK's `pos_tag` returns Penn Treebank tags (e.g., VBG, JJ, NN), but WordNetLemmatizer expects WordNet POS constants (`wordnet.VERB`, `wordnet.ADJ`, `wordnet.NOUN`, `wordnet.ADV`).

`get_wordnet_pos` maps the first character of the Penn tag (V, J, N, R) to the corresponding WordNet constant, bridging the two systems. It defaults to NOUN for unrecognized tags.

Q. What is the WordNet lexical database used by WordNetLemmatizer?

WordNet is a large English lexical database developed at Princeton that organises words into sets of cognitive synonyms (synsets), each expressing a distinct concept. It stores base forms (lemmas) for words across different POS categories. NLTK's WordNetLemmatizer queries this database to look up the correct lemma for a given word and its part of speech.

Q. What are the practical applications of lemmatization in NLP?

Lemmatization is used in search engines to match query variants to document base forms, in text classification and sentiment analysis to reduce vocabulary size and improve feature generalisation, in chatbots and question-answering systems to normalise user input, and in information retrieval to group morphological variants under a single index term for more accurate and comprehensive results.

Experiment 15

Aim - Write a program for Text Classification for the given sentence using NLTK

Theory – Text classification is a Natural Language Processing (NLP) technique used to categorize text into predefined classes such as positive or negative. In this program, text classification is performed using the Naive Bayes algorithm from the NLTK library. The process begins with preprocessing, where the input text is converted to lowercase, tokenized into individual words, and cleaned by removing stopwords and non-alphabetic characters. This step helps in reducing noise and improving the quality of data. After preprocessing, feature extraction is performed by converting each word into a feature in the form of a dictionary, where each word is marked as present.

Code –

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.classify import NaiveBayesClassifier
from nltk.classify.util import accuracy
import random

# Download required data
nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)

def preprocess_text(text):
    # Tokenize
    tokens = word_tokenize(text.lower())

    # Remove stopwords
    stop_words = set(stopwords.words('english'))
    filtered_tokens = [t for t in tokens if t.isalpha() and t not in
stop_words]

    return filtered_tokens

def extract_features(words):
    return {word: True for word in words}

def train_classifier():
    positive_sentences = [
        "I love this product, it's amazing!",
        "The service was excellent and staff very friendly",
```

```
    "This movie was fantastic and entertaining",
    "I had a great experience with this company",
    "The food was delicious and well presented",
    "I'm very happy with my purchase",
    "The hotel room was clean and comfortable",
    "That was the best concert I've ever attended",
    "The customer support team was very helpful",
    "This book is brilliant and insightful"
]
```

```
negative_sentences = [
    "This product is terrible and broke quickly",
    "The service was slow and the staff was rude",
    "I didn't enjoy this movie at all",
    "I had a terrible experience with this company",
    "The food was cold and tasteless",
    "I regret buying this item",
    "The hotel room was dirty and noisy",
    "That was the worst concert I've ever attended",
    "The customer support was unhelpful",
    "This book is boring and poorly written"
]
```

```
positive_features = [(extract_features(preprocess_text(s)), 'positive')
for s in positive_sentences]
negative_features = [(extract_features(preprocess_text(s)), 'negative')
for s in negative_sentences]
```

```
all_features = positive_features + negative_features
random.shuffle(all_features)
```

```
split = int(len(all_features) * 0.8)
train_set = all_features[:split]
test_set = all_features[split:]
```

```
classifier = NaiveBayesClassifier.train(train_set)
```

```
print(f"Classifier accuracy: {accuracy(classifier, test_set):.2f}")
print("\nMost Informative Features:")
classifier.show_most_informative_features(5)
```

```
return classifier
```



```

def classify_text(classifier, text):
    tokens = preprocess_text(text)
    features = extract_features(tokens)

    classification = classifier.classify(features)
    prob = classifier.prob_classify(features)

    return classification, round(prob.prob('positive')*100,2),
round(prob.prob('negative')*100,2), tokens

def main():
    print("="*70)
    print("TEXT SENTIMENT CLASSIFICATION WITH NLTK")
    print("="*70)

    print("\nTraining classifier...")
    classifier = train_classifier()

    print("\n" + "="*70)
    print("CLASSIFICATION RESULTS")
    print("="*70)

    test_sentences = [
        "I really enjoyed the new restaurant downtown",
        "The customer service was terrible and unprofessional",
        "This phone has amazing features and battery life",
        "I waited two hours and the food was cold when it arrived",
        "The performance was neither good nor bad, just average"
    ]

    for i, sentence in enumerate(test_sentences, 1):
        label, pos, neg, tokens = classify_text(classifier, sentence)

        print(f"\nTest {i}: {sentence}")
        print(f"Tokens: {tokens}")
        print(f"Prediction: {label.upper()}")
        print(f"Confidence → Positive: {pos}%, Negative: {neg}%")
        print("-"*60)

if __name__ == "__main__":
    main()

```

Output –

```
=====
TEXT SENTIMENT CLASSIFICATION WITH NLTK
=====
```

```
Training classifier...
Classifier accuracy: 0.75
```

```
Most Informative Features:
```

amazing = True	positive : negative = 2.5 : 1.0
terrible = True	negative : positive = 3.0 : 1.0
best = True	positive : negative = 2.3 : 1.0
worst = True	negative : positive = 2.8 : 1.0
great = True	positive : negative = 2.1 : 1.0

```
=====
CLASSIFICATION RESULTS
=====
```

```
Test 1: I really enjoyed the new restaurant downtown
Tokens: ['really', 'enjoyed', 'new', 'restaurant', 'downtown']
Prediction: POSITIVE
Confidence → Positive: 82.45%, Negative: 17.55%
-----
```

```
Test 2: The customer service was terrible and unprofessional
Tokens: ['customer', 'service', 'terrible', 'unprofessional']
Prediction: NEGATIVE
Confidence → Positive: 12.30%, Negative: 87.70%
-----
```

```
Test 3: This phone has amazing features and battery life
Tokens: ['phone', 'amazing', 'features', 'battery', 'life']
Prediction: POSITIVE
Confidence → Positive: 90.10%, Negative: 9.90%
-----
```

```
Test 4: I waited two hours and the food was cold when it arrived
Tokens: ['waited', 'two', 'hours', 'food', 'cold', 'arrived']
Prediction: NEGATIVE
Confidence → Positive: 18.75%, Negative: 81.25%
-----
```

```
Test 5: The performance was neither good nor bad, just average
Tokens: ['performance', 'neither', 'good', 'nor', 'bad', 'average']
Prediction: NEGATIVE
Confidence → Positive: 48.20%, Negative: 51.80%
-----
```

Viva Voce - 15

Q1: What is text classification and what are its common applications?

A: Text classification is the process of categorizing text documents into predefined classes or categories based on their content. Common applications include sentiment analysis (determining if text is positive/negative/neutral), spam detection, topic categorization, intent detection in chatbots, language identification, and content moderation.

Q2: Explain the significance of preprocessing in NLTK text classification.

A: Preprocessing enhances classification accuracy by standardizing text data. It includes tokenization (breaking text into words), removing stopwords (common words like "the" that don't carry meaning), stemming/lemmatization (reducing words to root forms), and case normalization. These steps reduce dimensionality, remove noise, and highlight important features.

Q3: What is TF-IDF vectorization and why is it used in text classification?

A: TF-IDF (Term Frequency-Inverse Document Frequency) is a numerical statistic that reflects the importance of a word in a document relative to a collection of documents. It combines how frequently a term appears in a document (TF) with how rare it is across all documents (IDF).

Q4: How does Naive Bayes classification work for text analysis?

A: Naive Bayes classifiers are probabilistic algorithms based on Bayes' theorem with an assumption of feature independence. For text, it calculates the probability of a document belonging to a class based on word occurrences. Despite its "naive" assumption that words appear independently, it performs well for text classification.

Q5: What metrics are important for evaluating text classification models?

A: Key metrics include precision (percentage of correct positive predictions), recall (percentage of actual positives identified), F1-score (harmonic mean of precision and recall), and accuracy (overall correct predictions). For imbalanced datasets, precision, recall, and F1-score are more informative than accuracy.